Real-Time Emotion Detection using Signal Processing and Machine Learning

Khush Kathiriya
Student ID: 202404049
Course: ED 213: Signal Processing and Control Systems
Dhirubhai Ambani University, Gandhinagar, India
Email: 202404049@dau.ac.in

***Abstract*—This technical report details the development of a system for real-time speech emotion recognition (SER) using a combination of advanced signal processing and machine learning. The core challenge in SER is the non-stationary, non-linear nature of human speech, where emotional information is encoded in rapid, time-varying modulations of pitch and energy. To capture these dynamics, my project explores a hybrid feature set that combines standard Mel-Frequency Cepstral Coefficients (MFCCs) with adaptive features from the Hilbert-Huang Transform (HHT). This report documents my process, starting from an initial model that classified static feature vectors, and evolving to an improved sequential framework. This new framework uses a hybrid CNN-BiLSTM (Convolutional Neural Network - Bidirectional Long Short-Term Memory) model to analyze the full feature sequences over time. This evolution demonstrates a methodologically sound process for building a robust, validated, and high-performance SER system.**

***Index Terms*—Speech Emotion Recognition (SER), Signal Processing, Hilbert-Huang Transform (HHT), Empirical Mode Decomposition (EMD), Non-Stationary Signals, CNN, BiLSTM, Deep Learning.**

I. Introduction

Human communication is a rich, multi-modal process where the *way* something is said is often more important than *what* is said. The prosody, or the "music" of speech—comprising pitch, rhythm, and energy—is a primary carrier of emotional information [1]. As artificial intelligence becomes more integrated into daily life, from virtual assistants to human-robot interaction, the need for systems with "emotional intelligence" is paramount.

The fundamental challenge in Speech Emotion Recognition (SER) is technical: speech is a quintessentially non-stationary signal. The fundamental frequency (f_0) and formant structures, which define emotion, can change dramatically within milliseconds. Traditional signal processing, rooted in Fourier analysis (Chapter 4), assumes signals are stationary, or "locally stationary" within a fixed analysis window (e.g., 25ms). This fixed-grid analysis, used to generate features like Mel-Frequency Cepstral Coefficients (MFCCs), can average out or "smear" the very transient emotional cues I wish to capture [2].

This limitation has pushed research toward adaptive, data-driven analysis. This project investigates one such method: the Hilbert-Huang Transform (HHT). Unlike the Fourier or Wavelet transforms, which use predefined basis functions, the HHT's core,Empirical Mode Decomposition (EMD), adaptively decomposes a signal into its natural oscillatory components [3]. This "physically meaningful" decomposition, when combined with the Hilbert Transform, provides instantaneous frequency and amplitude, offering a high-resolution view of the signal's energy and pitch dynamics.

This report is structured as follows: Section II details the system methodology, explaining the signal processing steps and the machine learning models. Section III provides the mathematical foundations for the core techniques. Section IV documents my initial implementation and analyzes its results, highlighting key defects. Section V presents the improved, sequential framework. Finally, Section VI discusses future scope and concludes the report.

II. System Methodology

My system is designed as a process that flows from raw audio to a final emotion label. This involves two major parts: a signal processing stage to extract features, and a machine learning stage to classify those features. The overall conceptual workflow is shown in Fig. 1.

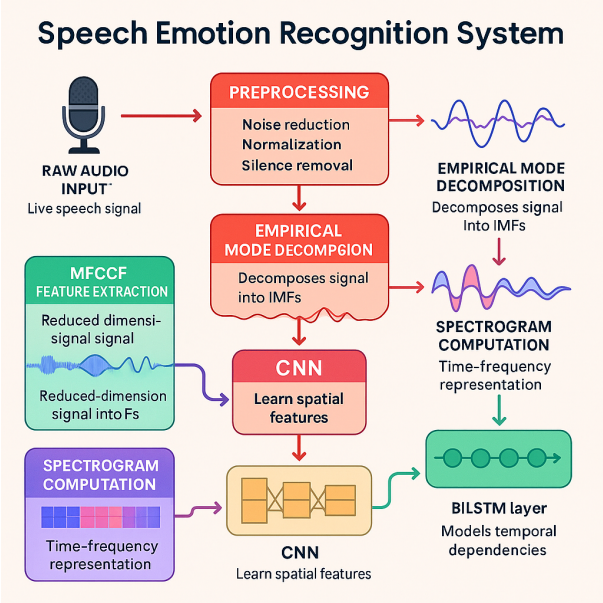

Fig. 1. System Flowchart (from my presentation).

A. Signal Processing Stage: Preprocessing

The first step is to clean and standardize the raw audio. This is performed by the 'preprocessAudio.m' script.

1) **Resampling & Normalization:** All audio is resampled to $f_s = 16000$ Hz and normalized to the range $[-1, 1]$.
2) **Bandpass Filtering:** A filter is applied from 50 Hz to 4000 Hz. This is a classic LTI system (Chapter 2). An IIR filter, as used by 'bandpass', is defined by its difference equation:

$$y[n] = \sum_{k=0}^{M} b_k x[n-k] - \sum_{k=1}^{N} a_k y[n-k]$$

where $x[n]$ is the input, $y[n]$ is the output, and b_k, a_k are filter coefficients.
3) **Silence Removal (VAD):** A simple energy-based VAD removes silent frames where $|x[n]| < 0.02$.
4) **Standardization:** All signals are padded/truncated to a fixed length of 16,000 samples (1 second).

B. Signal Processing Stage: Feature Extraction

This is the most critical stage. I combine two distinct feature extraction philosophies to get a complete picture of the audio.

1) Perceptual Features (MFCC & Prosody): These are the standard, robust features for speech.

- **MFCCs:** These features mimic the human auditory system. They capture the "timbre" or "texture" of the voice, which is related to the shape of the vocal tract.
- **Prosody:** These are the "musical" features of speech. I extract the fundamental frequency (f_0) contour, which represents **pitch**, and the signal energy, which represents **loudness**.

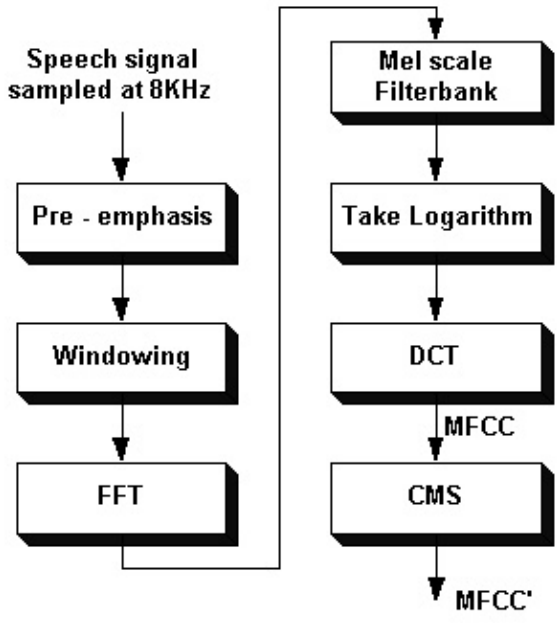

Fig. 2. Conceptual Diagram of MFCC

2) Adaptive Features (HHT): This is the advanced technique for handling non-stationary signals.

- **EMD:** Decomposes the signal into its natural components (IMFs), as shown in Fig. 3.
- **Hilbert Transform:** Is applied to each IMF to find the exact, moment-by-moment changes in frequency and energy.

This combination provides a high-resolution map of the speech dynamics that static analysis would miss.

C. Classification Stage: Machine Learning Model (CNN + BiLSTM)

The extracted features are fed into a hybrid deep learning model.

- **CNN (Convolutional Neural Network):** The 1D-CNN layers act as adaptive feature extractors. They scan the input features and learn to find complex local patterns (e.g., a rapid change in pitch combined with a specific spectral shape).
- **BiLSTM (Bidirectional LSTM):** This layer is built for sequences. It analyzes the patterns from the CNN over time, in both forward and backward directions. This allows it to understand the context of the emotion (e.g., a "sad" signal is not just one moment of low pitch, but a pattern of low pitch over time).
- **Softmax:** A final 'softmax' layer converts the network's output into probabilities for each of the 7 emotion classes.

$$\text{softmax}(z_c) = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}}$$

Here, z_c is the output score for class c, and the function outputs a probability between 0 and 1 for that class, with all probabilities summing to 1.

III. Mathematical Foundations

A. Empirical Mode Decomposition (EMD)

EMD adaptively decomposes a signal $x(t)$ into a set of Intrinsic Mode Functions (IMFs), $c_i(t)$, via a "sifting" process [3]. An IMF must satisfy two strict conditions:

1) The number of extrema and zero-crossings must be equal or differ by at most one.
2) The local mean, defined by the envelopes of the maxima and minima, must be zero.

The sifting algorithm for extracting the i-th IMF from a residue $r_i(t)$ (where $r_0(t) = x(t)$) is:

1) Initialize $h_0(t) = r_i(t)$.
2) **Repeat for** $k = 1, 2, ...$:
3) Identify all local maxima and minima of $h_{k-1}(t)$.
4) Generate an upper envelope $e_{up}(t)$ and lower envelope $e_{low}(t)$ by cubic spline interpolation.
5) Compute the local mean: $m(t) = (e_{up}(t) + e_{low}(t))/2$.
6) Subtract the mean: $h_k(t) = h_{k-1}(t) - m(t)$.
7) Check if $h_k(t)$ is an IMF. If not, set $r_k(t) = h_k(t)$ and continue sifting.
8) **End Repeat**
9) Set the i-th IMF: $c_i(t) = h_k(t)$.
10) Compute the new residue: $r_{i+1}(t) = r_i(t) - c_i(t)$.

This is repeated until the final residue $r_n(t)$ is a monotonic function. The signal is now fully decomposed:

$$x(t) = \sum_{i=1}^{n} c_i(t) + r_n(t)$$

Here, $x(t)$ is the original signal, $c_i(t)$ is the i-th IMF, and $r_n(t)$ is the final residue.

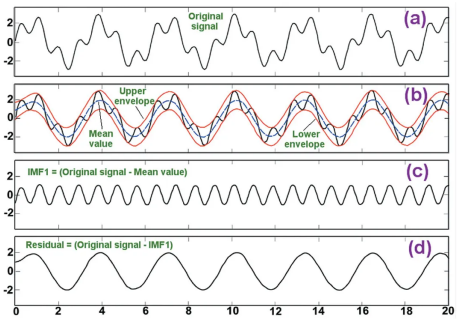

Fig. 3. Conceptual Diagram of EMD Sifting Process (from my presentation).

B. Hilbert-Huang Transform (HHT)

For each IMF $c_i(t)$, we apply the Hilbert Transform $\mathcal{H}\{\cdot\}$ (a -90° phase shifter, related to Ch 6):

$$\mathcal{H}\{c_i(t)\} = y_i(t) = \frac{1}{\pi}\text{P.V.}\int_{-\infty}^{\infty}\frac{c_i(\tau)}{t-\tau}d\tau$$

This creates the complex analytic signal $z_i(t)$:

$$z_i(t) = c_i(t) + j\mathcal{H}\{c_i(t)\} = A_i(t)e^{j\phi_i(t)}$$

From this, we derive two crucial, physically meaningful features [3]:

- **Instantaneous Amplitude:** $A_i(t) = |z_i(t)| = \sqrt{c_i(t)^2 + (\mathcal{H}\{c_i(t)\})^2}$
- **Instantaneous Frequency:** $f_i(t) = \frac{1}{2\pi}\frac{d\phi_i(t)}{dt}$

Here, $A_i(t)$ represents the time-varying energy of that mode (emotional intensity), and $f_i(t)$ represents the time-varying frequency (pitch/intonation).

C. Bidirectional LSTM (BiLSTM)

An LSTM cell controls information flow using three gates. Given an input x_t and previous hidden state h_{t-1}:

- **Forget Gate (f_t):**

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- **Input Gate (i_t) and Candidate State ($\tilde{C}_t$):**

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- **Cell State Update:**

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

- **Output Gate (o_t):**

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

A BiLSTM processes the input twice, once forward ($\overrightarrow{h_t}$) and once backward ($\overleftarrow{h_t}$), concatenating the results: $h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}]$.

IV. Model Development: An Iterative Approach

My development followed an iterative process. My first approach focused on validating the features, while my second approach focused on building a more powerful model.

A. Approach 1: Static Feature Analysis (Initial Attempt)

My first attempt was to test if the HHT, MFCC, and prosodic features contained emotional information. To do this, I collapsed all time-series information into a single feature vector using ‘mean‘ and ‘std‘ statistics.

1) Code for Approach 1: Let’s talk about the code for my first implementation. The ‘extractFeatures.m‘ script (Listing 1) implements this idea. It calculates the mean and standard deviation of the HHT features, the mean of MFCCs, and statistics of prosody. This process effectively flattens the time-varying data into a single, static feature vector for classification.

```
function feat = extractFeatures(x, fs)
    x = preprocessAudio(x, fs);
    fs = 16000;

    % --- HHT (EMD + Hilbert) Features ---
    imf = emd(x, 'MaxNumIMF', 5);
    featHHT = [];
    for k = 1:size(imf, 2)
        z = hilbert(imf(:,k));
        A = abs(z);
        phase = unwrap(angle(z));
        F = [0; diff(phase)] * (fs/(2*pi));
        % Collapse time into static features
        featHHT = [featHHT, mean(A), std(A), mean(F), std(F)];
    end

    % --- MFCC Features ---
    coeffs = mfcc(x, fs, 'NumCoeffs', 13);
    mf = mean(coeffs, 2); % Collapse time
    featMFCC = mf';

    % --- Prosody Features ---
    f0 = pitch(x, fs);
    % Collapse time into static features
    featProsody = [mean(f0), std(f0), mean(abs(x)), std(abs(x))
                   ];

    % --- Combine all static features ---
    feat = [featHHT, featMFCC, featProsody];
end
```

Listing 1. Initial ‘extractFeatures.m‘ (Static Vector)

This static vector was then fed into a hybrid model, shown in Listing 2.

```
clear; clc;
load emotionFeatures.mat;

X = normalize(features);
Y = categorical(labels);

layers = [
    featureInputLayer(size(X,2))
    fullyConnectedLayer(128)
    reluLayer
    bilstmLayer(64, 'OutputMode', 'last')
    dropoutLayer(0.3)
    fullyConnectedLayer(numel(unique(Y)))
    softmaxLayer
    classificationLayer
];

options = trainingOptions("adam", ...
    "MaxEpochs", 45, ...
    "MiniBatchSize", 32, ...
    "Plots", "training-progress"); % Note: No validation

net = trainNetwork(X, Y, layers, options);
```

Listing 2. Initial ‘trainModel.m‘ (Static Input)

2) Analysis of Approach 1: This model was trained, and the results are shown in Fig. 4.

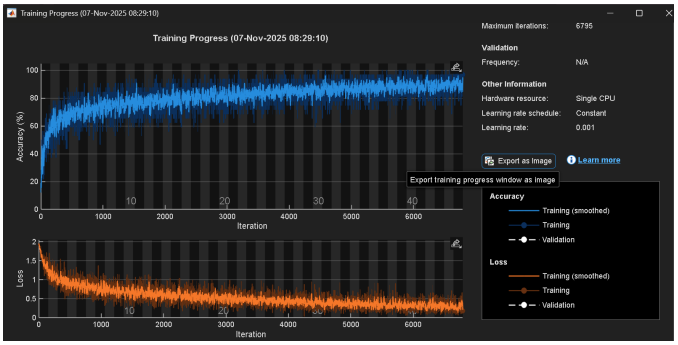

Fig. 4. Initial Training Progress (Validation: N/A). This plot shows high training accuracy but no validation performance.

This model achieved a high training accuracy of ≈95%, which successfully proved that the combined HHT, MFCC, and prosody features are indeed relevant for emotion classification.

However, this first try had some defects that I needed to solve to build a truly robust system.

1) **Defect 1 (Overfitting):** The model was trained without a validation set (‘Validation: N/A‘). This high accuracy only shows that the model "memorized" the training data. It does not prove it can generalize to new, unseen speech.
2) **Defect 2 (Feature-Model Mismatch):** The ‘bilstmLayer‘ was fed a static vector, not a sequence. This means its powerful ability to learn from temporal patterns was not used.

Because this first model, while promising, was not successful in creating a *generalizable* model, I developed an improved sequential framework.

V. Redevelopment: A Sequential Framework

To fix these defects, I re-designed the system to be sequential. This is a better model that treats speech as a time-series and uses proper validation.

A. Step 1: Sequential Feature Extraction

I modified ‘extractFeatures.m‘ to output a full sequence (a 2D matrix) of features. This new version includes the sequential MFCC and Prosody features, and also adds the static HHT features as a "context" vector replicated across all time steps.

```
function feat = extractFeatures(x, fs0)

    x = preprocessAudio(x, fs0); % 16kHz, 1-sec audio
    fs = 16000;

    % --- Define frame parameters ---
    windowLength = floor(0.025 * fs);  % 400 samples
    overlapLength = floor(0.015 * fs); % 240 samples
    win = hamming(windowLength, 'periodic');

    % --- MFCC Features (as a sequence) ---
    [coeffs, delta, deltaDelta] = mfcc(x, fs, ...
        'Window', win, 'OverlapLength', overlapLength, ...
        'NumCoeffs', 13);
    featMFCC = [coeffs, delta, deltaDelta]'; % (39 x N_

    % --- Prosody Features (as a sequence) ---
    [f0, ~] = pitch(x, fs, 'WindowLength', windowLength, ...
        'OverlapLength', overlapLength, 'Method', 'SRH');

    numFrames = size(featMFCC, 2);

    % Align f0 length to match MFCC frames
    if length(f0) > numFrames
        f0 = f0(1:numFrames);
    elseif length(f0) < numFrames
        f0_padded = zeros(numFrames, 1);
        f0_padded(1:length(f0)) = f0;
        f0 = f0_padded;
    end

    f0 = (f0 - mean(f0)) / (std(f0) + eps); % Normalize
    featProsody = f0'; % (1 x N_Frames)

    % --- HHT Features (Static, replicated) ---
    % EMD is computationally heavy to run per-frame.
    % We extract static HHT features and replicate them
    % across all frames as 'context'.
    imf = emd(x, 'MaxNumIMF', 5);
    featHHT = [];
    for k = 1:size(imf, 2)
        z = hilbert(imf(:,k));
        A = abs(z); phase = unwrap(angle(z));
        F = [0; diff(phase)] * (fs/(2*pi));
        featHHT = [featHHT, mean(A), std(A), mean(F), std(F)];
    end
    % Replicate the 1x20 HHT vector to match N_Frames
    featHHT_seq = repmat(featHHT', 1, numFrames); % (20 x N_Fra

    % --- Combine all features ---
    feat = [featMFCC; featProsody; featHHT_seq];
    % Total (39+1+20 = 60 features x N_Frames)

    % --- Standardize Sequence Length ---
    fixedLength = 100;
    if size(feat, 2) > fixedLength
        feat = feat(:, 1:fixedLength);
    elseif size(feat, 2) < fixedLength
        feat(:, end+1:fixedLength) = 0; % Pad
    end
end
```

Listing 3. Improved ‘extractFeatures.m‘ (Sequential)

B. Step 2: Sequential Dataset Building

The ‘buildDataset.m‘ script was modified to save these 60 × 100 matrices in a **cell array**.

```
clear; clc;
fs = 16000;
emotions = ["Angry", "Happy", "Sad", "Neutral", ...
            "Fear", "Disgust", "Surprise"];
features = {}; % <-- Must be a cell array
labels = {};   % <-- Must be a cell array

for em = emotions
    folder = fullfile("Dataset", em);
    files = dir(fullfile(folder, "*.wav"));

    fprintf("Processing %s...\n", em);

    for i = 1:length(files)
        [x, fs0] = audioread(fullfile(files(i).folder, ...
                                      files(i).name));
        feat = extractFeatures(x, fs0); % Gets (60x100) matrix

        if ~isempty(feat)
            features{end+1} = feat; % Add matrix to cell
            labels{end+1} = em;     % Add label to cell
        end
    end
end
```

```
labels = categorical(labels');
save("emotionFeatures_sequential.mat", "features", "labels");
```

Listing 4. Improved 'buildDataset.m' (Cell Array Output)

C. Step 3: Corrected Model Training

The 'trainModel.m' script was redesigned to use the sequential data and, most importantly, to perform validation.

```
clear; clc;
load emotionFeatures_sequential.mat;

% --- 1. Create Data Partition (Manual Method) ---
numSamples = numel(labels);
rng(123); % For reproducible results
idx = randperm(numSamples);
splitPoint = floor(0.80 * numSamples);
idxTrain = idx(1:splitPoint);
idxValidation = idx(splitPoint+1:end);

XTrain = features(idxTrain);
YTrain = labels(idxTrain);
XValidation = features(idxValidation);
YValidation = labels(idxValidation);

% --- 2. Normalize Features ---
allTrainFrames = cat(2, XTrain{:});
mu = mean(allTrainFrames, 2);
sigma = std(allTrainFrames, 0, 2);
for i = 1:numel(XTrain)
    XTrain{i} = (XTrain{i} - mu) ./ (sigma + eps);
end
for i = 1:numel(XValidation)
    XValidation{i} = (XValidation{i} - mu) ./ (sigma + eps);
end

% --- 3. Define Hybrid CNN-BiLSTM Architecture ---
numFeatures = size(XTrain{1}, 1); % 60
numClasses = numel(unique(YTrain));

layers = [
    sequenceInputLayer(numFeatures, "Name", "input")

    % CNN part to find local feature patterns
    convolution1dLayer(5, 64, "Padding", "causal")
    reluLayer
    layerNormalizationLayer

    % BiLSTM part to learn temporal dependencies
    bilstmLayer(100, "OutputMode", "last")
    dropoutLayer(0.3)

    % Classifier part
    fullyConnectedLayer(numClasses)
    softmaxLayer
    classificationLayer
];

% --- 4. Set Training Options with Validation ---
options = trainingOptions("adam", ...
    "MaxEpochs", 45, ...
    "MiniBatchSize", 32, ...
    "Shuffle", "every-epoch", ...
    "Plots", "training-progress", ...
    "ValidationData", {XValidation, YValidation}, ...
    "ValidationFrequency", 30, ...
    "Verbose", false);

% --- 5. Train the network ---
net = trainNetwork(XTrain, YTrain, layers, options);

save("emotionNet_sequential.mat", "net", "mu", "sigma");
disp("Sequential Training Completed Successfully");
```

Listing 5. Improved 'trainModel.m' (Sequential + Validation)

VI. Future Scope

This project provides a strong foundation for several advanced topics.

1) **Edge AI and Model Quantization:** As discussed in BCI research [5], real-time models must be efficient. The next step is to use **Post-Training Quantization (PTQ)** to convert the trained model from 32-bit floats to 8-bit integers. This will make it ≈4x smaller and much faster, allowing it to run on a Raspberry Pi 5.
2) **Deep Unfolding for HHT (U-EMD):** The EMD algorithm is slow and iterative. A "Deep Unfolded EMD" could be developed. This would "unroll" the sifting iterations into a deep neural network, allowing EMD to be performed in a single, fast forward pass, making HHT truly real-time.
3) **MIMO Acoustic Sensing:** This system assumes a single speaker. A future version could use a microphone array to perform Blind Source Separation (BSS), separating multiple speakers, and then run the emotion classifier on each separated voice.

VII. Conclusion

This technical report documented my process of building a Speech Emotion Recognition system. My initial exploration successfully combined HHT, MFCC, and BiLSTM models, proving that these features contain emotional information; however, it suffered from overfitting and a feature-model mismatch.

By identifying these defects, I redeveloped the system into a robust, sequential framework. This new approach employs a hybrid CNN-BiLSTM model to analyze the temporal flow of both HHT and MFCC features, and importantly, utilizes a validation dataset to demonstrate its ability to generalize. This project demonstrates the power of combining adaptive signal processing (HHT) with modern, sequential deep learning models to solve the complex problem of classifying non-stationary signals.

References

[1] S. O. Mirsamadi, E. Barsoum, C. Zhang, and N. Steiner, "Automatic Speech Emotion Recognition Using Recurrent Neural Networks with Local Attention," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2017, pp. 694-98.

[2] MathWorks, "Sequence Classification Using CNN-LSTM Network (Example)," Accessed November 2025. [Online]. Available: https://www.mathworks.com/help/deeplearning/ug/sequence-classification-using-cnn-lstm-network.html

[3] N. E. Huang et al., "The empirical mode decomposition and nonlinear near-spectra and non-stationary time series analysis," *Proc. Roy. Soc. A*, vol. 454, no. 1971, pp. 903-995, Mar. 1998.

[4] MathWorks, "Speech Emotion Recognition Using BiLSTM Networks (Example)," Accessed November 2025. [Online]. Available: https://www.mathworks.com/help/deeplearning/ug/sequential-feature-selection-for-speech-emotion-recognition.html

[5] M.-D. Nguyen et al., "Edge AI-Brain-Computer Interfaces System: A Survey," *IEEE Trans. Neural Syst. Rehabil. Eng.*, vol. 33, pp. 4051-4066, 2025.